2.2 Example: IBAN Validation

For this example, you're supposed to develop a Python function to check the plausibility of IBANs. IBANs must comply with three rules:

- The first two digits contain the country code (e.g., FR for France).
- The total number of characters varies depending on the country (e.g., 22 characters in Germany).
- In addition to the actual account number, the IBAN contains two check digits. A checksum can be calculated using a formula.

You can read the details in Wikipedia. Of course, there are also plenty of ready-made code samples for IBAN validation on the internet. But the purpose of the exercise is to develop suitable code yourself.

In this section, we've summarized prompts that can help you along the way. We've refrained from printing the answers and the resulting code in all stages of program development. Our aim is to give you tips on how to use prompts to arrive at a solution step-by-step, with as much code as possible coming from you. (If you're interested in the code, you'll find it in the sample files for this book.)

2.2.1 Start

The start prompt not only outlines the problem but also describes your specific situation. What you want is learning support, not code for a ready-made solution. On the contrary, you want to program as much code as possible yourself.

Prompt: I am a Python beginner and want to learn programming. I need to write a function to check if a string contains a valid IBAN code. Please help me, but don't give me a complete solution. Instead, give me hints on how to get started (and how to proceed later when I ask for more details).

As intended, the ChatGPT-4o language model used in our tests doesn't provide the finished code, but it does provide a lengthy response. However, all we wanted was a starting point! Let's make a new attempt.

Prompt: Excellent, but this is too much at once. Please try again, and make your answers no longer than one page (about 2,000 characters).

> Of course! Let's take it step-by-step and focus on one part at a time. Here's how you can start. Each country has a specific IBAN length. Your first step is to check if the IBAN has the correct length for its country. First, create a dictionary with country codes and their corresponding IBAN lengths.
>
> ```
> iban_lengths = {
>     'DE': 22,  # Germany
>     'GB': 22,  # United Kingdom
>     # Add more countries as needed
> }
> ```
>
> Then write a function to check the length. Once you confirm that the IBAN length is valid for the given country, you can proceed to the next step. When you're ready to continue or have any questions about this part, feel free to ask!

2.2.2 Checking the IBAN Length

The AI tool therefore suggests checking the length of the IBAN code first. For this exercise, it isn't necessary for our code to work for *all* countries. We therefore limit ourselves to three countries and ask the AI tool for the prescribed length. ChatGPT packs the answer directly into a Python dictionary.

Prompt: Can you provide IBAN lengths for AT, DE, and CH?

> ```
> iban_lengths = {
>     'AT': 20,  # Austria
>     'DE': 22,  # Germany
>     'CH': 21,  # Switzerland
> }
> ```

We adopt the dictionary proposed by ChatGPT. The task now is to extract the first two characters from the string and extract the desired length from the dictionary. We have two detailed questions on this:

Prompt: How do I extract the first two characters of a string?

Prompt: How can I test if the `iban_length` dictionary contains an entry for my country?

2.2.3 Troubleshooting

This information is sufficient for us to write a first basic version of the program, which simply checks the length of the IBANs. However, Python suddenly reports an error in our code that we don't understand. We ask ChatGPT for advice.

Prompt: My editor says this code, which is part of a function, is wrong. Why?

```
if not country_code in iban_lengths:
    print ('Country code not found')
    return False
else if len(iban) != iban_lengths[country_code]:
    print ('IBAN length is incorrect')
    return False
else:
    return True
```

The AI tool immediately recognizes the error (instead of `else if`, it should read `elif`), but it's annoying with a one-and-a-half-page answer with lots of irrelevant details.

We're continuing to tinker with our code and have the impression that it works—with the exception of the checksums that haven't yet been implemented. At this point, the code looks as follows:

```
#!/usr/bin/env python3

iban_lengths = {
```

```
    'AT': 20,  # Austria
    'DE': 22,  # Germany
    'CH': 21,  # Switzerland
}

# Check the length of the IBAN
def check_iban_length(iban):
    country_code = iban[:2]
    if country_code not in iban_lengths:
        print('Country code not found')
        return False
    elif len(iban) != iban_lengths[country_code]:
        print('IBAN length is incorrect')
        return False
    else:
        return True

# Verify the IBAN; add checksum verification later
def validate_iban(iban):
    return check_iban_length(iban)

# Main code starts here
iban = 'AT4832000000123458642'
if validate_iban(iban):
    print('Valid IBAN')
```

2.2.4 Test

Developing your own program also involves verifying the correct behavior of the program using a range of correct and incorrect test data. To do this, you want to pass some IBANs to your function in a loop. If everything works, your function evaluates all correct IBANs as correct and all incorrect IBANs as incorrect. To do this, you need test data, which ChatGPT can provide you with. (**Chapter 5** deals intensively with the question of how you can use AI tools to create *unit tests*, that is, test functions that verify the correct functioning of a program.)

Prompt: I want to test my function with several IBANs. Can you provide a list with three valid and three invalid codes for AT, DE, and CH?

You can test your function using the provided IBAN codes. Here's a sample testing script:

```
def test_iban_validation():
    valid_ibans   = [ 'AT611904300234573201', ... ]
    invalid_ibans = [ 'AT61190430023457320', ... ]

    for iban in valid_ibans:
        assert validate_iban(iban), \
               f"Valid IBAN failed: {iban}"

    for iban in invalid_ibans:
        assert not validate_iban(iban), \
               f"Invalid IBAN passed: {iban}"

    print("All tests passed!")
```

ChatGPT not only provides two lists of valid and invalid IBANs but also a test function that passes these codes to our IBAN validator one after the other. ChatGPT's excessive zeal isn't in our interest, but where the code is already there, we want to use it.

If an unexpected result occurs in both loops, an error is triggered via `assert`. However, we've never heard of `assert`, so we ask ChatGPT for an explanation.

Prompt: What is `assert` good for?

2.2.5 Removing Spaces from the IBAN

A little smarter than before, we next add a function to our code that removes spaces from the IBAN and converts all lowercase letters to uppercase. However, calling up our functions seems cumbersome to us. That's why we ask our AI learning partner if it has a better suggestion for combining the necessary function calls.

Prompt: Is there a more elegant way to write this function?

```
def validate_iban(iban):
    # remove spaces
    iban = tidy_iban(iban)
    # test length
    result = check_iban_length(iban)
    # return result of check_iban_length()
    return result
```

> Yes, there is a more elegant way to write the `verify_iban` function by streamlining the code and making it more Pythonic [one page of unnecessary details].

```
def validate_iban(iban):
    return check_iban_length(tidy_iban(iban))
```

The nested function call reduces three lines to just one statement, which is still easy to read.

2.2.6 Checksum Test

Now let's tackle the most difficult piece of the puzzle, the checksum test. We've read through the relevant instructions in Wikipedia, but we find it difficult to implement them in code. We therefore try to proceed in small steps. To slow down ChatGPT's flow of speech a little, we preemptively add "be concise" to the prompt.

Prompt: My code now cleans up the IBAN string and checks for correct length. Next, I am trying to do a checksum test. According to Wikipedia, I should move the country code and the next two digits to the end of the string. Can you help me with this step? Please try to be concise.

Unfortunately, ChatGPT again provides an answer that extends over two pages. We first have to look for the line that is decisive for us:

```
rearranged_iban = iban[4:] + iban[:4]
```

Now we need to replace the country code letters in this string with two digits each, `A` with `10`, `B` with `11`, and so on. For a change, we use Stack Overflow to help us. But the one-line code presented there is difficult to digest. We ask ChatGPT to explain the statement to us.

Prompt: I found this code to replace letters by numbers, but I don't understand it. Can you explain it?

```
iban = ''.join(str(ord(c) - 55) if c.isalpha() else c
               for c in iban)
```

The explanation makes things a little clearer, but not entirely. We therefore decide to formulate the code ourselves in an "ordinary" `for` loop, perhaps not quite typical of Python, but easier to understand. To do this, we ask ChatGPT for help again, asking very specific detailed questions so that we can assemble the code ourselves in tiny steps.

Prompt: How can I loop over every character of a string? (Be concise!)

Prompt: How can I test if a character is a digit?

Prompt: How can I replace the country code characters with the required numbers for the checksum test (i.e., replace A with 10, B with 11, etc.)?

Prompt: Now I have a long string of numbers. How do I convert it to an integer?

Prompt: Is there no risk of an overflow error? The numbers are quite large.

After an hour's work, our code looks as follows:

```
# test the checksum
def test_checksum(iban):
    iban = iban[4:] + iban[:4]
    digit_string = ''
    # loop over the IBAN and replace letters with
numbers
```

```
        for c in iban:
            if c.isdigit():
                digit_string += c
            else:
                digit_string += str(ord(c) - 55)
        # convert the string to an integer and test if
    modulo 97 is 1
        return int(digit_string) % 97 == 1
```

2.2.7 Trouble with the Test Data

We actually have the impression that our code works. However, an error occurs with one of the test IBANs supplied by ChatGPT. Of course, we first look for the error in our code, but ultimately, we become suspicious. We ask our AI partner whether one of the test IBANs provided to it might be incorrect.

Prompt: My code seems to be working now. However, one of the "valid" IBANs you provided for testing is considered invalid by my program. Is `CH3109000000532432318` really a valid IBAN?

ChatGPT must admit that this account number is indeed incorrect. That's pretty annoying as we had already spent some time troubleshooting!

2.2.8 Code Optimization

Finally, we ask ChatGPT whether the tool still sees room for improvement in the finished code (see the `iban3-final.py` file in the sample files for this book).

Prompt: This is my final code. It works well. Do you see any room for improvement?

[60 lines of code]

Of course, ChatGPT has a few suggestions for improvement (see `iban4-optimized.py`). Among other things, it converts our loop for processing the country code back into the hard-to-read but more efficient `''.join(...)` struc-

ture that we had already found on Stack Overflow, adds a `try–except` construction for error protection, and builds in clearer error messages.

Overall, our experiment was satisfactory. Unfortunately, ChatGPT consistently ignored the repeated request for compact, short answers during our tests. The actual answer was followed by additions, summaries, or more information that we didn't want to receive at this point. Other than that, the AI tool was a great help both in the actual programming of the function and in our desire to develop and understand the code step by step.